

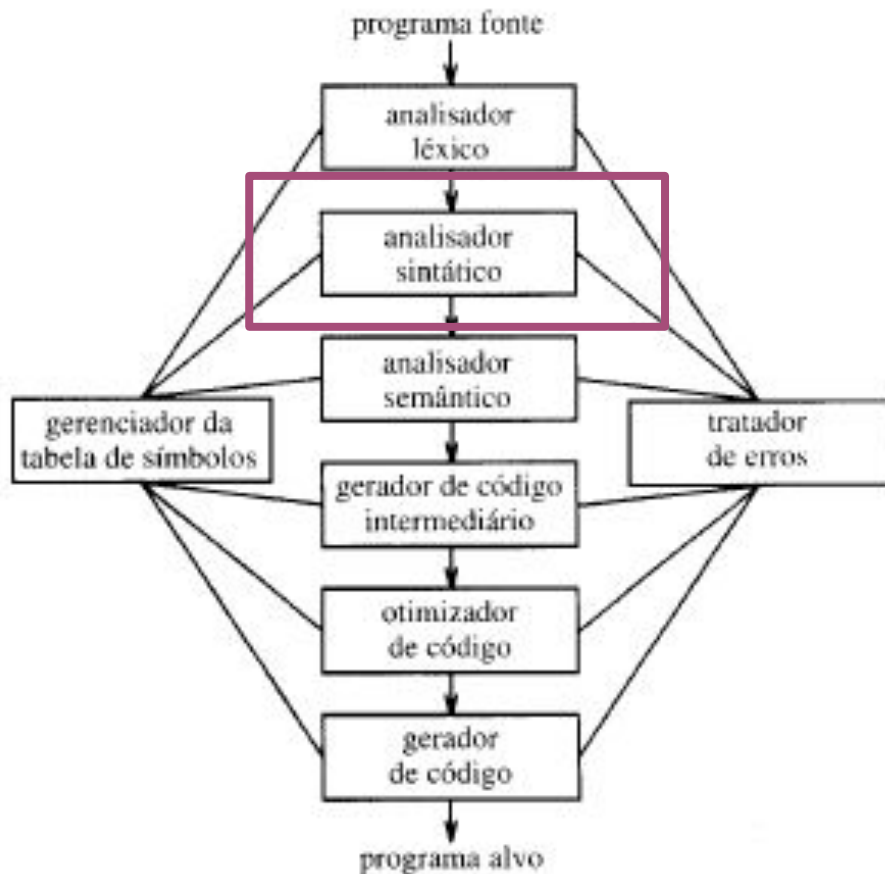
Compiladores

Juliana da Costa Feitosa
juliana.feitosa@unesp.br
Docente UNESP - Bauru-SP

Aula 6

Análise Sintática

Introdução



Introdução

- O problema da **análise sintática** para uma dada gramática livre de contexto, consiste em descobrir, para uma cadeia x qualquer, se a cadeia pertence ou não pertence à linguagem da gramática

Introdução

- No caso afirmativo, queremos adicionalmente descobrir a maneira (ou uma das maneiras) pela qual a cadeia pode ser derivada seguindo as regras da gramática.

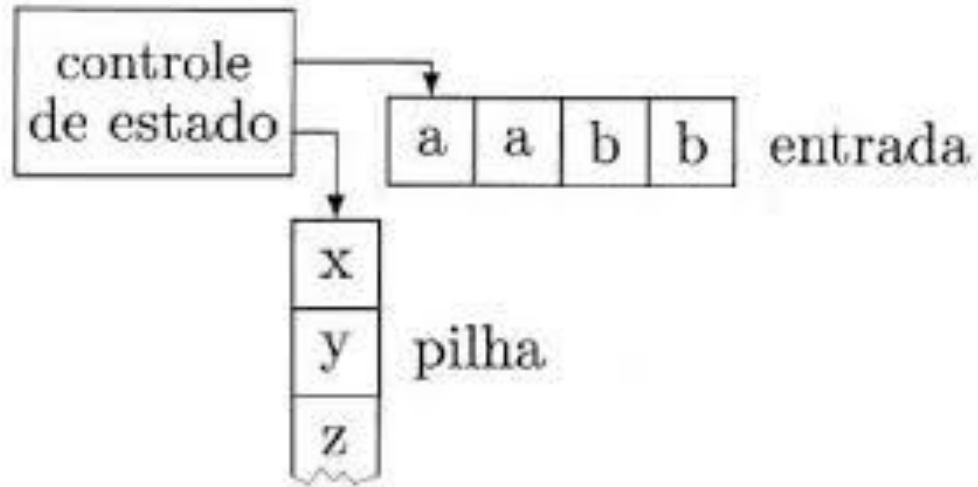
Introdução

- O **analisador sintático** utiliza os primeiros componentes dos **átomos (tokens)** produzidos pelo analisador léxico para criar uma representação intermediária

Autômato com pilha

- Reconhecem as linguagens geradas por **GLC (Gramática Livre de Contexto)**
- Esses autômatos são extensões dos **Autômatos Finitos**
- Possui
 - Uma pilha ou uma memória que funciona como pilha
 - Uma fila com os símbolos de entrada

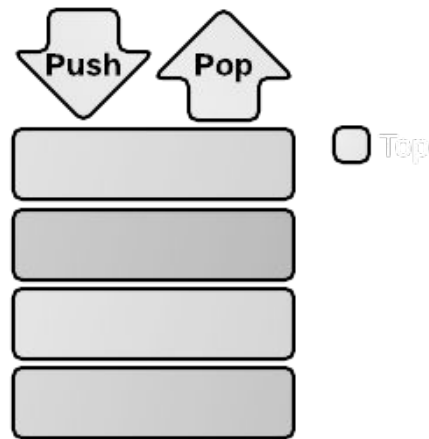
Autômato com pilha



Autômato com pilha

- Pilha

- O único acesso permitido para a pilha é o **topo**
 - A inserção é feita pelo topo da pilha
 - O último elemento que foi inserido será o primeiro a ser removido



Autômato com pilha não determinístico

- Conhecido também como **APND**
- M é um APND representado por $(E, V, P, f, q_0, z_0, F)$, onde:
 - **E**: conjunto finito de estados
 - **V**: alfabeto ou símbolos de entrada (aparece na fila e pertence a Σ^*). São os elementos de que são formadas as cadeias de entrada submetidas ao autômato para aceitação

Autômato com pilha não determinístico

- M é um APND representado por $(E, V, P, f, q_0, z_0, F)$, onde:
 - **P**: alfabeto ou símbolos finito da pilha (a pilha pode conter apenas símbolos de P , porém, P e V podem ter símbolos em comum). São os códigos armazenados pelo autômato em sua memória auxiliar.
 - **f**: função de transição/movimentação do autômato finito à pilha

Autômato com pilha não determinístico

- M é um APND representado por $(E, V, P, f, q_0, z_0, F)$, onde:
 - q_0 : é o estado inicial
 - z_0 : é o símbolo inicial da pilha (pertencente a $P \cup \{\lambda\}$)
 - F : é o conjunto de estados finais (estados de aceitação)

Autômato com pilha não determinístico

- **Exemplo:** $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$
 - $E = \{q_0, q_1\}$
 - $V = \{0, c, 1\}$
 - $P = \{0, 1\}$
 - $f = f$
 - $q_0 = q_0$
 - $z_0 = \lambda$
 - $F = \{q_1\}$

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$
 - Seja s conjunto de símbolos da pilha, inclusive λ
 - $f(q_0, 0, s) = (q_0, 0s)$
 - $f(q_0, 1, s) = (q_0, 1s)$
 - $f(q_0, c, s) = (q_1, s)$
 - $f(q_1, 1, 1) = (q_1, \lambda)$
 - $f(q_1, 0, 0) = (q_1, \lambda)$

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$
 - Seja s conjunto de símbolos da pilha, inclusive λ
 - $f(q_0, 0, s) = (q_0, 0s)$ ←
 - $f(q_0, 1, s) = (q_0, 1s)$
 - $f(q_0, c, s) = (q_1, s)$
 - $f(q_1, 1, 1) = (q_1, \lambda)$
 - $f(q_1, 0, 0) = (q_1, \lambda)$

q_0 : estado atual

0: símbolo lido como entrada

s : símbolo no topo da pilha

q_1 : estado seguinte

0s: push 0/escrever 0 por cima do símbolo no topo da pilha

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0				
---	--	--	--	--

λ



0
 λ

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

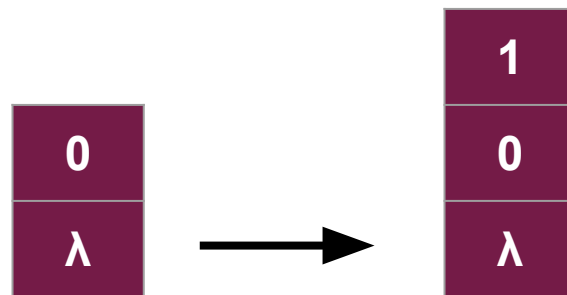
- **$f(q_0, 1, s) = (q_0, 1s)$**

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1			
---	---	--	--	--



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

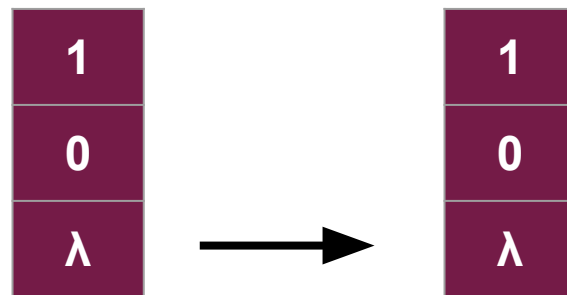
- $f(q_0, 1, s) = (q_0, 1s)$

- **$f(q_0, c, s) = (q_1, s)$**

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	c		
---	---	---	--	--



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

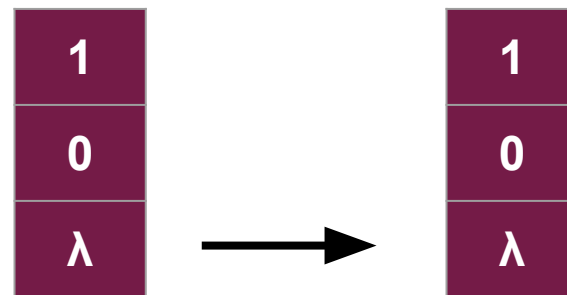
- $f(q_0, 1, s) = (q_0, 1s)$

- **$f(q_0, c, s) = (q_1, s)$**

- ~~$f(q_1, 1, 1) = (q_1, \lambda)$~~

- ~~$f(q_1, 0, 0) = (q_1, \lambda)$~~

0	1	c		
---	---	---	--	--



Percebam que
agora fomos para
o estado q_1

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

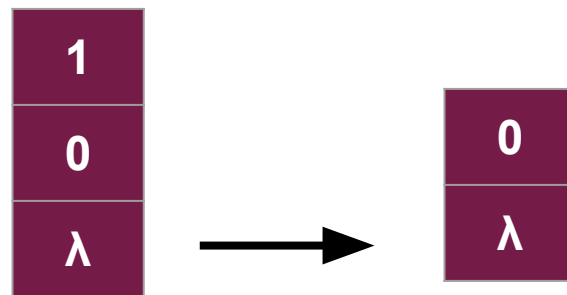
- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- **$f(q_1, 1, 1) = (q_1, \lambda)$**

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	c	1	
---	---	---	---	--



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

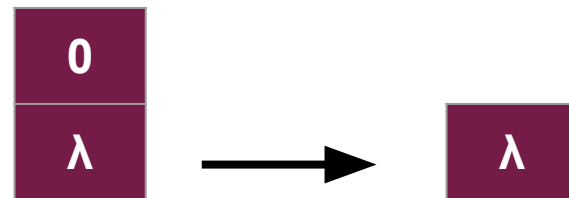
- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	c	1	0
---	---	---	---	---



Autômato com pilha não determinístico

- Podemos reconhecer uma linguagem de três formas:
 - **Não pertence a linguagem:** palavra processada, pilha vazia, não chegou ao estado final
 - **Não pertence a linguagem:** palavra processada, pilha não vazia, chegou ao estado final
 - **Pertence a linguagem:** palavra processada, pilha vazia e chegou ao estado final

Autômato com pilha não determinístico

- Exemplo:

- $S \rightarrow 0S0$

- $S \rightarrow 1S1$

- $S \rightarrow c$

- A cadeia $w = 0101c1010$ é derivável de S

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0								
---	--	--	--	--	--	--	--	--

λ



0
 λ

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

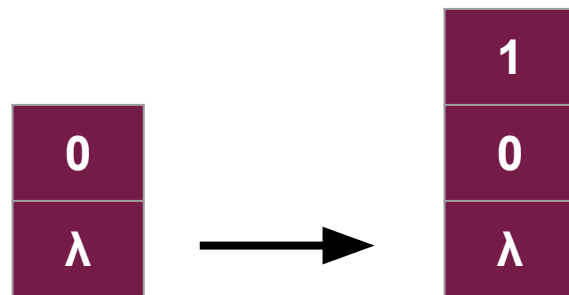
- **$f(q_0, 1, s) = (q_0, 1s)$**

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1							
---	---	--	--	--	--	--	--	--



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

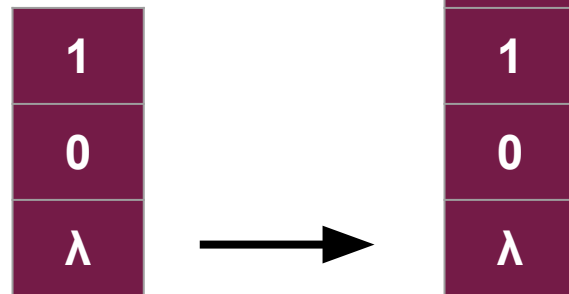
- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	0						
---	---	---	--	--	--	--	--	--



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

- **$f(q_0, 1, s) = (q_0, 1s)$**

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	0	1					
---	---	---	---	--	--	--	--	--

0
1
0
λ



1
0
1
0
λ

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

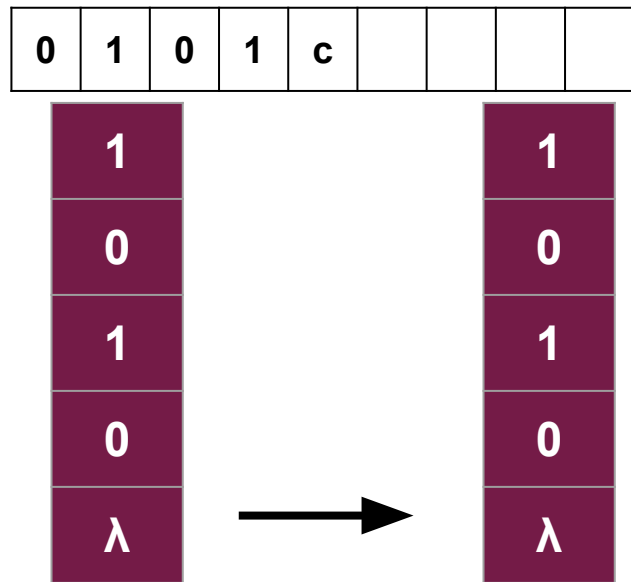
- $f(q_0, 0, s) = (q_0, 0s)$

- $f(q_0, 1, s) = (q_0, 1s)$

- **$f(q_0, c, s) = (q_1, s)$**

- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- **$f(q_1, 1, 1) = (q_1, \lambda)$**

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	0	1	c	1			
---	---	---	---	---	---	--	--	--

1
0
1
0
λ



0
1
0
λ

Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

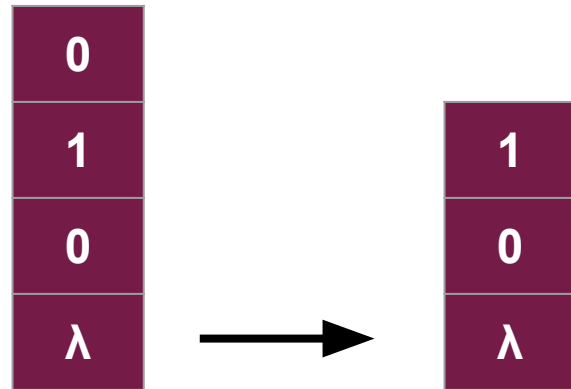
- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- $f(q_1, 1, 1) = (q_1, \lambda)$

- **$f(q_1, 0, 0) = (q_1, \lambda)$**

0	1	0	1	c	1	0		
---	---	---	---	---	---	---	--	--



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

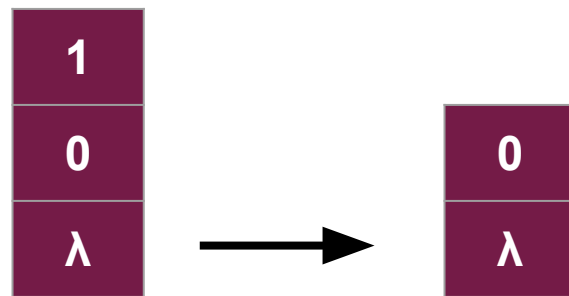
- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

- **$f(q_1, 1, 1) = (q_1, \lambda)$**

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	0	1	c	1	0	1	
---	---	---	---	---	---	---	---	--



Autômato com pilha não determinístico

- Exemplo: $M = (\{q_0, q_1\}, \{0, c, 1\}, \{0, 1\}, f, q_0, \lambda, \{q_1\})$

- $f(q_0, 0, s) = (q_0, 0s)$

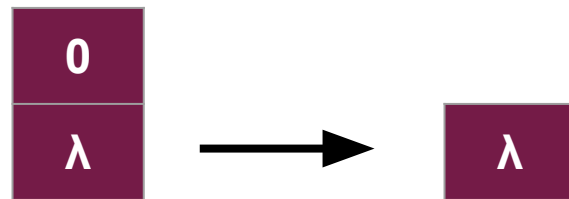
- $f(q_0, 1, s) = (q_0, 1s)$

- $f(q_0, c, s) = (q_1, s)$

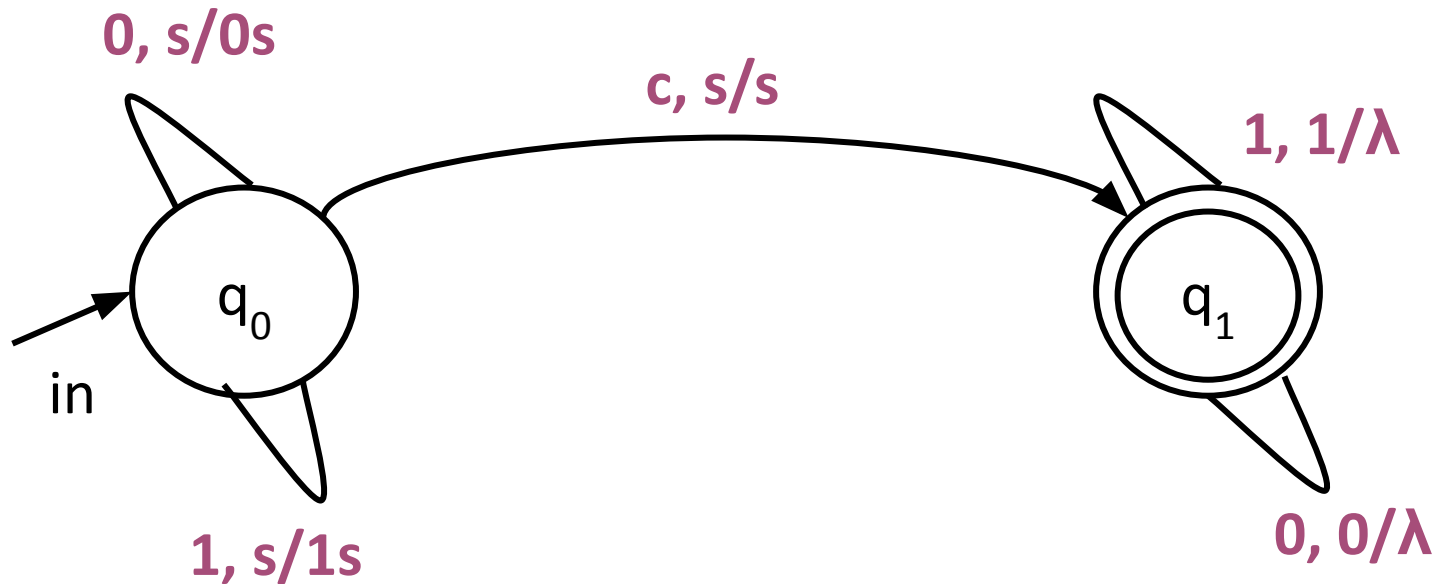
- $f(q_1, 1, 1) = (q_1, \lambda)$

- $f(q_1, 0, 0) = (q_1, \lambda)$

0	1	0	1	c	1	0	1	0
---	---	---	---	---	---	---	---	---



Autômato com pilha não determinístico



Árvores de derivação

- Considera-se uma **árvore de derivação** (ou **árvore sintática**) para G se:
 - Todo nó tem um valor, que é um símbolo de $V \cup \{\lambda\}$,
 $V = V_N \cup V_T$
 - O valor da raiz é S
 - Se um nó é interno e tem valor A , então A precisa pertencer a V_N
 - Pois terá regra e é uma GLC

Árvores de derivação

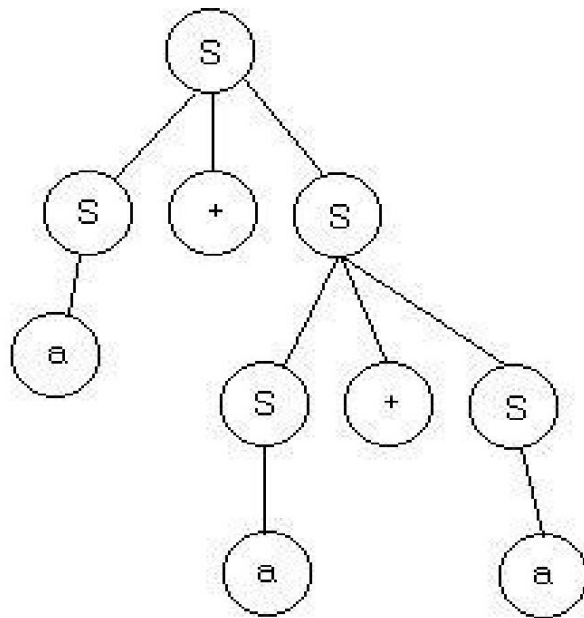
- Considera-se uma **árvore de derivação** (ou árvore sintática) para G se:
 - Se um nó n tem valor A e os nós $n_1, \dots, n_k$ são filhos do nó n da esquerda para a direita, com valores $x_1, \dots, x_k$ respectivamente, então a produção $A \rightarrow x_1, \dots, x_k$ deve pertencer a P (**regras de G**).
 - Se o nó n tem valor λ , então n é uma folha e é único filho do seu pai

Árvores de derivação

- **Exemplo:** Seja $G = (V_N, V_T, P, S)$
 - $V_N : \{S\}$
 - $V_T : \{a, +\}$
 - $P : \{(S \rightarrow S+S), (S \rightarrow a)\}$
 - $S : S$

Árvores de derivação

- Exemplo:
 - Árvore de derivação para **w = a+a+a**



Árvores de derivação

- Teorema

- Seja $G = (V_N, V_T, P, S)$ uma gramática do tipo **GLC**
- A palavra w é derivável a partir de S em G , se e somente se, existe uma árvore de derivação em G que produz w

Ambiguidade

- Quando utiliza-se gramática para definir uma linguagem de programação, deve-se certificar que tal gramática não seja **ambígua**
- Caso contrário, o programador e o compilador podem ter opiniões diferentes sobre o significado de alguma sentença

Ambiguidade

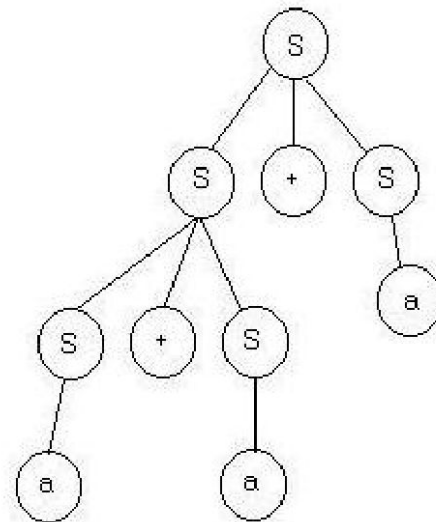
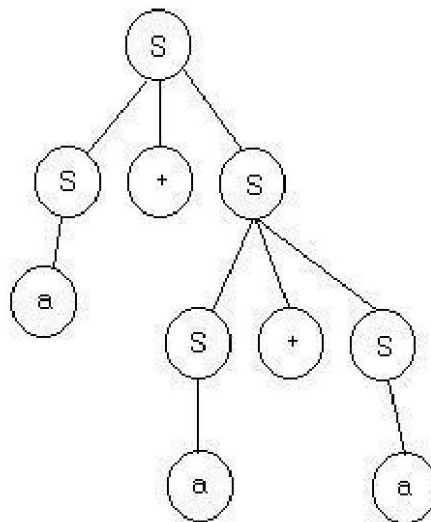
- Uma gramática $G = (V_N, V_T, P, S)$, do tipo GLC é **ambígua** se existe uma palavra **w** gerada por **G**, onde **w** pode ser produzida por **duas ou mais árvores** de derivação distintas

Ambiguidade

- **Exemplo:** Considere a gramática G com a seguinte regra de produção (exemplo anterior)
 - $S \rightarrow S + S \mid a$
- **Esta gramática é ambígua?**
- **Para saber,** propõe-se uma cadeia w que pode ser gerada por duas árvores de derivação distintas

Ambiguidade

- Exemplo:
 - Para **w = a+a+a**

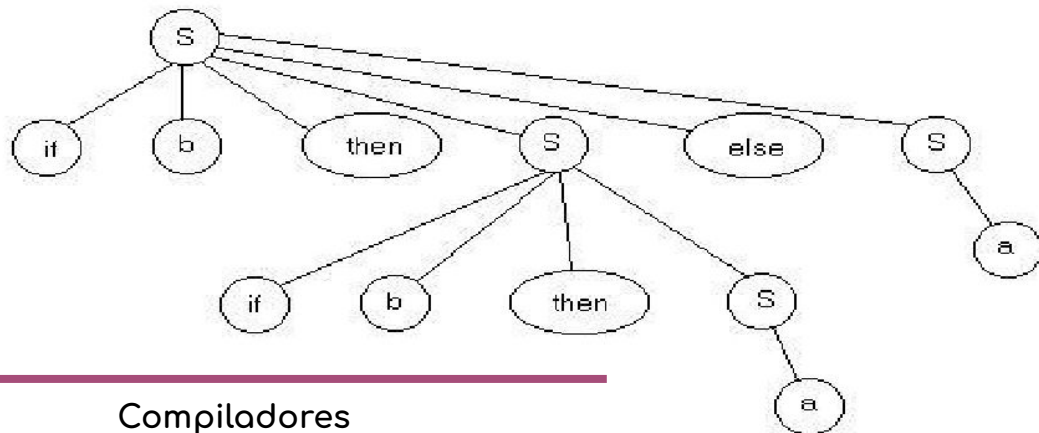
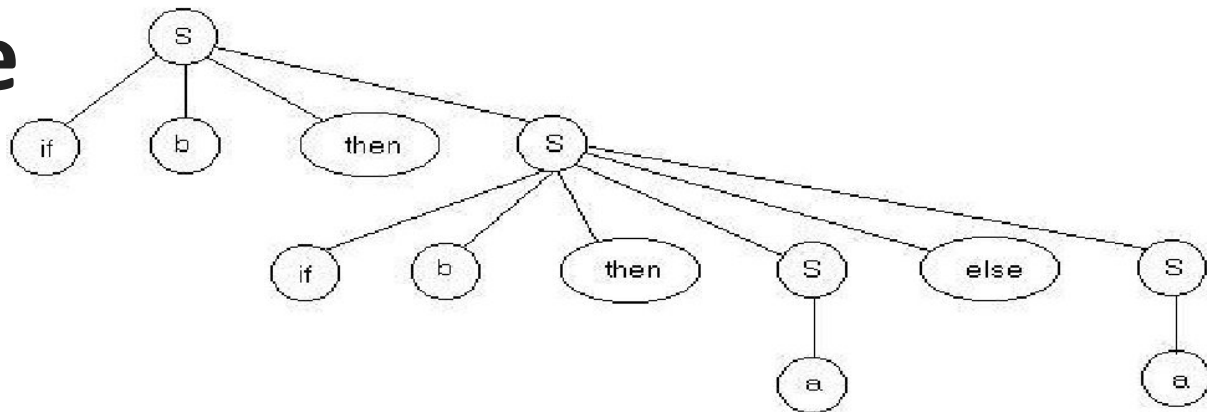


Ambiguidade

- **Exemplo 2:** Considere a gramática G com a seguinte regra de produção
 - $S \rightarrow \text{if } b \text{ then } S$
 - $S \rightarrow \text{if } b \text{ then } S \text{ else } S$
 - $S \rightarrow a$
- **Esta gramática é ambígua?**
- **Para saber,** propõe-se uma cadeia w que pode ser gerada por duas árvores de derivação distintas

Ambiguidade

- Exemplo: Para
 $w = \text{if } b \text{ then if } b \text{ then } a \text{ else } a$



Ambiguidade

- Não é possível escrever um algoritmo que, dada uma gramática arbitrária, **determine se ela é ou não ambígua**
- Algumas gramáticas ambíguas podem ter outras gramáticas não ambíguas equivalentes (com regras de produção diferentes, por exemplo)

Linguagem inerentemente ambígua

- Uma linguagem é inerentemente ambígua se toda gramática que a gera for ambígua
 - **Exemplo:**
 - $L(G) : \{ a^n b^n c^m \mid n, m \geq 0 \} \cup \{ a^n b^m c^n \mid n, m \geq 0 \}$
 - $L(G) = L1 \cup L2$
 - $L1(G) = \{ a^n b^n c^m \mid n, m \geq 0 \}$
 - $L2(G) = \{ a^n b^m c^n \mid n, m \geq 0 \}$

Linguagem inerentemente ambígua

- Exemplo: $L(G) : L1(G) \cup L2(G)$

- $S \rightarrow S1 \mid S2$

- $S1 \rightarrow XC \mid \epsilon$

- $X \rightarrow aXb \mid \epsilon$

- $C \rightarrow cC \mid \epsilon$

- $S2 \rightarrow AY \mid \epsilon$

- $A \rightarrow aA \mid \epsilon$

- $Y \rightarrow bYc \mid \epsilon$

Linguagem inerentemente ambígua

- Exemplo: $L(G) : L1(G) \cup L2(G)$

- $S \rightarrow S1 \mid S2$

- $S1 \rightarrow XC \mid \varepsilon$

- $X \rightarrow aXb \mid \varepsilon$

- $C \rightarrow cC \mid \varepsilon$

- $S2 \rightarrow AY \mid \varepsilon$

- $A \rightarrow aA \mid \varepsilon$

- $Y \rightarrow bYc \mid \varepsilon$

- Para $w = abc$

- $S \rightarrow S1 \rightarrow XC \rightarrow aXbC \rightarrow abC \rightarrow abcC \rightarrow abc$

- $S \rightarrow S2 \rightarrow AY \rightarrow aAY \rightarrow aY \rightarrow abYc \rightarrow abc$

Linguagem inerentemente ambígua

- Exemplo: $L(G) : L1(G) \cup L2(G)$

- $S \rightarrow S1 \mid S2$

- $S1 \rightarrow XC \mid \varepsilon$

- $X \rightarrow aXb \mid \varepsilon$

- $C \rightarrow cC \mid \varepsilon$

- $S2 \rightarrow AY \mid \varepsilon$

- $A \rightarrow aA \mid \varepsilon$

- $Y \rightarrow bYc \mid \varepsilon$

- $L(G)$ é inerentemente ambígua porque G é ambígua, já que a cadeia **abc** tem duas derivações distintas, sendo uma de $S1$ e outra de $S2$

Eliminação da ambiguidade

- A eliminação da ambiguidade deve ser feita alterando as regras de produção, conseguindo assim outra gramática equivalente
- **Exemplo:** Considere a gramática G com a seguinte regra de produção
 - $S \rightarrow S + S \mid a$

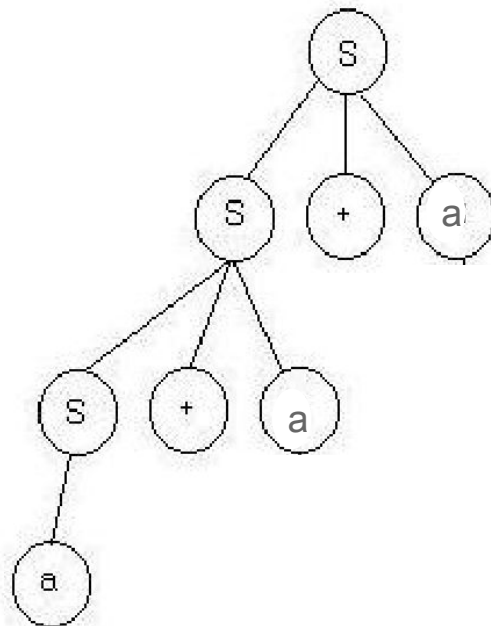
■ Solução a esquerda

- $S \rightarrow S + a \mid a$

Fixo um lado

Eliminação de ambiguidade

- Exemplo:
 - Para **w = a+a+a**



Eliminação da ambiguidade

- **Exemplo 2:** Considere a gramática G com a seguinte regra de produção
 - $S \rightarrow S + S \mid S * S \mid a$
 - **Solução**
 - $S \rightarrow S + P \mid P$
 - $P \rightarrow P * a \mid a$

Gramáticas reduzidas

- Restrições em gramáticas
 - Nenhuma gramática deverá conter a regra $U ::= U$
 - As gramáticas também podem conter regras supérfluas, que nunca são empregadas

Gramáticas reduzidas

- As gramáticas também podem conter regras supérfluas, que nunca são empregadas
 - **Exemplo:**
 - $Z ::= Be$
 - $A ::= Ae \mid e$
 - $C ::= Cf$
 - $D ::= f$
- 
- A regra D nunca é chamada

Gramáticas reduzidas

- Um símbolo não terminal U deve satisfazer duas condições
 - Aparecer em alguma forma sentencial
 - Conseguimos chegar em sentenças a partir de U
 - No exemplo anterior a regra C não cumpre a condição pois nunca chega a um terminal
 - $C ::= Cf$

Ascendente e Descendente

- Os métodos de **análise sintática** podem ser classificados segundo a maneira pela qual a árvore de derivação da cadeia analisada x é construída

Ascendente e Descendente

- Nos **métodos descendentes**, a árvore de derivação correspondente a x é construída de cima para baixo, ou seja, da raiz (o símbolo inicial S) para às folhas, onde se encontra x .

Ascendente e Descendente

- Nos **métodos ascendentes**, a árvore de derivação correspondente a x é construída de baixo para cima, ou seja, das folhas, onde se encontra x , para a raiz, onde se encontra o símbolo inicial S

Análise sintática descendente

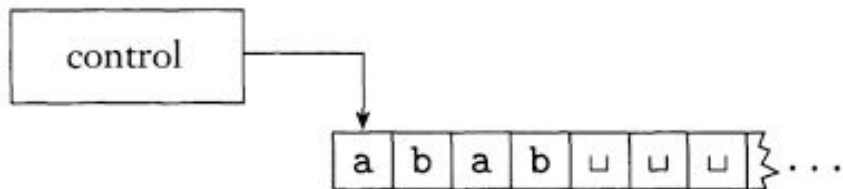
- Utilizaremos **gramáticas livre de contexto, reduzidas e não ambíguas**, que serão apresentadas em notação BNF apenas com as regras de produção
- Nos algoritmos de análise descendente, a construção começa pela **raiz e segue na direção das folhas**

Máquina de Turing

- Quando iniciamos uma **Máquina de Turing (MT)**, podemos ter **três resultados** ao processar a **palavra w**:
 - *Aceita*
 - *Rejeita*
 - *Entra em loop*
- Pode ser difícil definir se uma MT está em loop ou apenas demorando para processar uma palavra
- **Decisores**: MTs que **sempre param** (aceitou ou rejeitou)

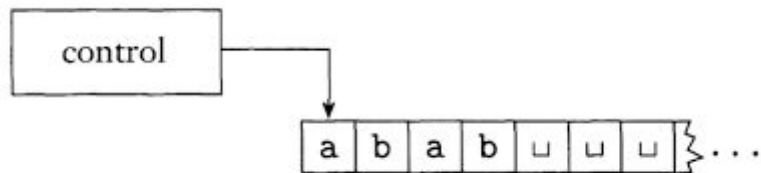
Máquina de Turing

- Neste modelo possuímos uma **fita infinita** e um **controle finito**.
- A máquina possui uma cabeça de leitura e escrita que pode ler/escrever símbolos na fita e movimentar-se sobre ela, tanto para a **direita** quanto para a **esquerda**
- Inicialmente, a fita contém apenas a cadeia de entrada

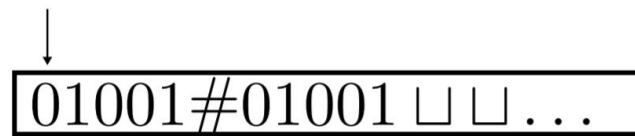


Máquina de Turing

- Se a máquina precisa armazenar alguma informação, ela pode escrever algo na fita.
- Para acessar uma informação, basta posicionar a cabeça na posição correta e ler aquele símbolo



Máquina de Turing



- **Exemplo:** Considere uma Máquina de Turing M1 para testar a pertinência na linguagem:
 - $B = \{w#w \mid w \in \{0, 1\}^*\}$
 - Aqui temos uma linguagem cujas palavras são seguidas por elas próprias e separadas por cerquilha (#).
- Não podemos acessar cada símbolo através de acesso aleatório, mas podemos andar de um lado para o outro da cerquilha e comparar os caracteres (acesso sequencial)

Máquina de Turing

↓
01001#01001 □ □ ...

- Solução:
 - Faça uma varredura na entrada para assegurar que ela contém uma única ocorrência do símbolo #. Se não, **rejeite**.
 - Faça um zigue-zague na fita para casar os símbolos nos dois lados da separação pelo #. Se os símbolos casam, marque eles com um 'x', caso contrário, **rejeite**.
 - Quando todos os símbolos da esquerda forem marcados com um 'x', verifique se existe algum símbolo diferente de 'x' e □ à direita. Se resta algum, **rejeite**, caso contrário, **aceite**.

Máquina de Turing

- **Definição formal:**

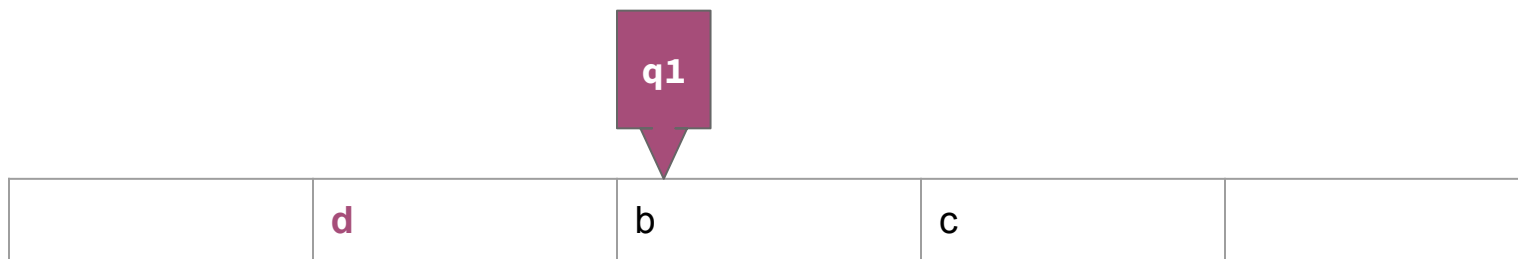
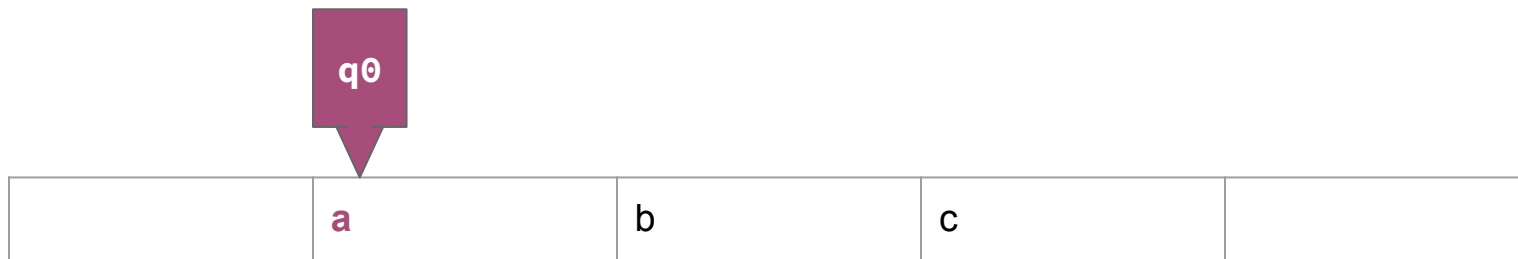
- O coração de uma MT é uma função de transição δ .
- Dado um estado e um símbolo na posição apontada pela cabeça, δ mapeia em outro estado, escreve um símbolo na fita de acordo com a posição da cabeça e move para esquerda ou para direita.
 - **Exemplo:** $\delta(q, a) = (r, b, L)$
 - A MT escreve o símbolo **b** no lugar de **a**, muda do estado **q** para o estado **r** e move a cabeça uma posição para a esquerda (**L**)

Máquina de Turing

- Uma MT é uma **7-upla** $(Q, \Sigma, \Gamma, \delta, q_0, q_{aceita}, q_{rejeita})$, em que Q , Σ e Γ são conjuntos finitos.
 - **Q** : o conjunto finito de estados.
 - **Σ** : o alfabeto finito de entrada, que não contém o símbolo $\sqcup$.
 - **Γ** : o alfabeto finito da fita, onde $\sqcup \in \Gamma$ e $\Sigma \subseteq \Gamma$.
 - **δ** : $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
 - **$q_0 \in Q$** : é o estado inicial
 - **$q_{aceita} \in Q$** : é o estado de aceitação
 - **$q_{rejeita} \in Q$** : é o estado de rejeição

Máquina de Turing

- Seja a função de transição $\delta(q_0, a) = (q_1, d, R)$.



Máquina de Turing

Seja $M = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, 0, \square\}, \delta, q_0, \{q_4\})$ com:

$\delta(q_0, 1) = (q_0, 1, R)$,

$\delta(q_0, 0) = (q_1, 1, R)$,

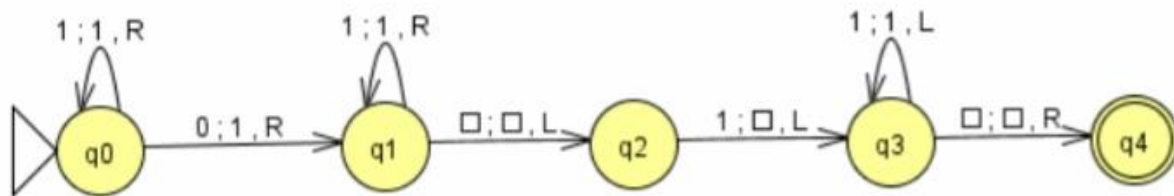
$\delta(q_1, 1) = (q_1, 1, R)$,

$\delta(q_1, \square) = (q_2, \square, L)$,

$\delta(q_2, 1) = (q_3, 0, L)$,

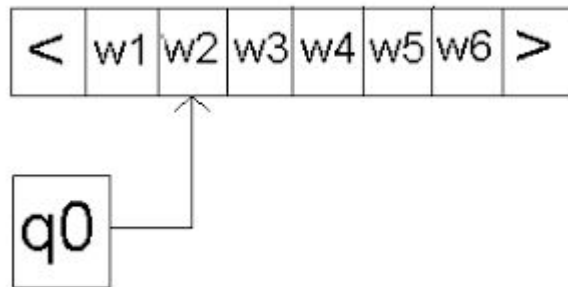
$\delta(q_3, 1) = (q_3, 1, L)$,

$\delta(q_3, \square) = (q_4, \square, R)$.



Autômatos Linearmente Limitados

- São Máquinas de Turing com **fita limitada** usadas para reconhecer **linguagens sensíveis ao contexto**
- O funcionamento é idêntico ao das MTs
- Possui dois delimitadores
 - **<: início da palavra na fita**
 - **>: final da palavra na fita**



Autômatos Linearmente Limitados

- O cursor de leitura/escrita **não pode** mover para fora desses delimitadores
- Só consegue resolver problemas que requer uma **memória proporcional à usada para entrada**
- A palavra w só será aceita quando chegarmos no estado final

Autômatos Linearmente Limitados

